

Requirements Quality, Testability, and Traceability

This lab gives students practical experience in applying requirements quality and testability principles to a small real-world software problem. Students will learn how to move from an unclear problem description to clear user stories, measurable requirements, acceptance criteria, implementation, tests, and traceability evidence.

The purpose of the lab is to show that software quality begins before coding. If requirements are unclear, incomplete, or difficult to test, the resulting software may still fail to meet stakeholder needs even if the code works correctly. Students will practise improving vague requirements, implementing the improved requirements in C# using Visual Studio, and writing tests that provide evidence that the requirements have been satisfied.

The lab also introduces students to the practical role of a Requirements Traceability Matrix (RTM). Students will link requirements to acceptance criteria and test cases so they can see which requirements have evidence, which areas may have gaps, and what needs to be reviewed if a requirement changes. This supports quality assurance, change management, regression testing, and release decision-making.

GitHub Copilot will be used as an AI-assisted learning and development tool during the lab. Students may use Copilot to generate test ideas, review code, suggest edge cases, or improve validation logic. However, students must critically review all AI-assisted outputs, modify them where necessary, test them, and be able to explain their final solution.

Library Book Reservation System

A small community library currently manages book reservations manually using emails and spreadsheets. This causes quality problems such as duplicate reservations, unclear reservation status, missing member details, and difficulty checking whether a book is available.

The library wants a simple software prototype that allows a member to reserve a book if the book is available and the member provides valid details. The prototype should prevent invalid reservations and provide clear feedback.

User Story

As a library member, I want to reserve an available book so that I can borrow it when I visit the library.

Initial Vague Requirement

The system should allow users to reserve books easily.

Improved Testable Requirements

Requirement ID	Requirement
REQ-LIB-01	The system shall allow a member to reserve a book only when the book is available.
REQ-LIB-02	The system shall reject a reservation if the member ID is empty.
REQ-LIB-03	The system shall reject a reservation if the book is already reserved.
REQ-LIB-04	The system shall return a clear success or failure message for every reservation attempt.

Acceptance Criteria

Acceptance Criterion ID	Acceptance Criterion
AC-01	Given an available book and a valid member, when the member reserves the book, then the reservation succeeds.
AC-02	Given an empty member ID, when a reservation is attempted, then the reservation fails with a clear message.
AC-03	Given a book that is already reserved, when another reservation is attempted, then the reservation fails.
AC-04	Every reservation attempt returns a message that explains the result.

Create the Visual Studio Solution

Step 1: Create the Main Project

1. Open Visual Studio.
2. Create a new project.
3. Select *Class Library*.
4. Name the project: *LibraryReservation*
5. Name the solution: *LibraryReservation*
6. Delete the default *Class1.cs* file.

Step 2: Create the Test Project

1. Right-click the solution.
2. Select *Add > New Project*.
3. Select *MSTest Test Project*.
4. Name the test project: *LibraryReservation.Tests*
5. Add a project reference:
 - Right-click the test project *Dependencies*.
 - Select *Add Project Reference*.
 - Tick *LibraryReservation*.
 - Click *OK*.

Step 3: Create the Book Class

Create a file named: Book.cs

Add the following code:

```
namespace LibraryReservation
{
    public class Book
    {
        public string Id { get; }
        public string Title { get; }
        public bool IsReserved { get; private set; }

        public Book(string id, string title)
        {
            if (string.IsNullOrEmpty(id))
                throw new ArgumentException("Book ID is required.");

            if (string.IsNullOrEmpty(title))
                throw new ArgumentException("Book title is required.");

            Id = id;
            Title = title;
            IsReserved = false;
        }

        public void MarkAsReserved()
        {
            if (IsReserved)
                throw new InvalidOperationException("Book is already reserved.");

            IsReserved = true;
        }
    }
}
```

Step 4: Create the Member Class

Create a file named: Member.cs

Add the following code:

```
namespace LibraryReservation
{
    public class Member
    {
        public string Id { get; }
        public string FullName { get; }
```

```

public Member(string id, string fullName)
{
    if (string.IsNullOrEmpty(id))
        throw new ArgumentException("Member ID is required.");

    if (string.IsNullOrEmpty(fullName))
        throw new ArgumentException("Member name is required.");

    Id = id;
    FullName = fullName;
}
}
}

```

Step 5: Create the ReservationResult Class

Create a file named: ReservationResult.cs

Add the following code:

```

namespace LibraryReservation
{
    public class ReservationResult
    {
        public bool Success { get; }
        public string Message { get; }

        public ReservationResult(bool success, string message)
        {
            Success = success;
            Message = message;
        }
    }
}

```

Step 6: Create the ReservationService Class

Create a file named: ReservationService.cs

Add the following code:

```

namespace LibraryReservation
{
    public class ReservationService
    {
        public ReservationResult ReserveBook(Book book, Member member)
        {
            if (book == null)

```

```

        return new ReservationResult(false, "Reservation failed: book details are required.");
    };

    if (member == null)
        return new ReservationResult(false, "Reservation failed: member details are required.");

    if (book.IsReserved)
        return new ReservationResult(false, $"Reservation failed: '{book.Title}' is already reserved.");

    book.MarkAsReserved();

    return new ReservationResult(true, $"Reservation successful: '{book.Title}' has been reserved for {member.FullName}.");
}
}
}

```

Build the solution and fix any errors.

Use GitHub Copilot

Step 7: Ask Copilot for Test Ideas

Open GitHub Copilot Chat and use this prompt:

Suggest MSTest unit tests for this C# library reservation system. Focus on testable requirements, acceptance criteria, edge cases, and clear expected results.

Review the suggestions. Do not copy blindly. Select useful ideas and modify them to match the requirements.

Record in your notes:

- Prompt used
- One useful suggestion
- One suggestion you modified
- One suggestion you rejected
- Why human judgement was required

Write Tests Linked to Requirements

Step 8: Create Reservation Tests

In the test project, create a file named: ReservationServiceTests.cs

Add the following tests:

```

using Microsoft.VisualStudio.TestTools.UnitTesting;
using LibraryReservation;

```

```
namespace LibraryReservation.Tests
{
    [TestClass]
    public class ReservationServiceTests
    {
        [TestMethod]
        public void ReserveBook_AvailableBookAndValidMember_ReturnsSuccess()
        {
            var book = new Book("B001", "Software Testing Basics");
            var member = new Member("M001", "Aroha Smith");
            var service = new ReservationService();

            ReservationResult result = service.ReserveBook(book, member);

            Assert.IsTrue(result.Success);
            StringAssert.Contains(result.Message, "Reservation successful");
        }

        [TestMethod]
        public void ReserveBook_AvailableBook_MarksBookAsReserved()
        {
            var book = new Book("B001", "Software Testing Basics");
            var member = new Member("M001", "Aroha Smith");
            var service = new ReservationService();

            service.ReserveBook(book, member);

            Assert.IsTrue(book.IsReserved);
        }

        [TestMethod]
        public void Member_EmptyMemberId_ThrowsException()
        {
            Assert.ThrowsException<ArgumentException>(() =>
                new Member("", "Aroha Smith"));
        }

        [TestMethod]
        public void ReserveBook_AlreadyReservedBook_ReturnsFailure()
        {
            var book = new Book("B001", "Software Testing Basics");
            var member1 = new Member("M001", "Aroha Smith");
            var member2 = new Member("M002", "John Chen");
```

```
var service = new ReservationService();

service.ReserveBook(book, member1);
ReservationResult result = service.ReserveBook(book, member2);

Assert.IsFalse(result.Success);
StringAssert.Contains(result.Message, "already reserved");
}

[TestMethod]
public void ReserveBook_NullBook_ReturnsClearFailureMessage()
{
    var member = new Member("M001", "Aroha Smith");
    var service = new ReservationService();

    ReservationResult result = service.ReserveBook(null, member);

    Assert.IsFalse(result.Success);
    StringAssert.Contains(result.Message, "book details are required");
}

[TestMethod]
public void ReserveBook_NullMember_ReturnsClearFailureMessage()
{
    var book = new Book("B001", "Software Testing Basics");
    var service = new ReservationService();

    ReservationResult result = service.ReserveBook(book, null);

    Assert.IsFalse(result.Success);
    StringAssert.Contains(result.Message, "member details are required");
}
}
```

Run all tests and confirm that they pass.

Create a Requirements Traceability Matrix

Step 9: Create a Documentation Folder

Create a folder named:

docs

Inside the folder, create a file named:

RTM.md

Add the following Requirements Traceability Matrix:

Requirement ID	Requirement Summary	Acceptance Criteria	Test Case	Status
REQ-LIB-01	Reserve only available book	AC-01	ReserveBook_AvailableBookAndValidMember_ReturnsSuccess; ReserveBook_AvailableBook_MarksBookAsReserved	Passed
REQ-LIB-02	Reject empty member ID	AC-02	Member_EmptyMemberId_ThrowsException	Passed
REQ-LIB-03	Reject already reserved book	AC-03	ReserveBook_AlreadyReservedBook_ReturnsFailure	Passed
REQ-LIB-04	Return clear success or failure message	AC-04	ReserveBook_NullBook_ReturnsClearFailureMessage; ReserveBook_NullMember_ReturnsClearFailureMessage	Passed

Add a short paragraph:

Traceability helps the team check whether each requirement has test evidence. It also supports change management because if a requirement changes, the related test cases can be identified, reviewed, and updated.

Managing Requirement Change

Step 10: Apply a Requirement Change

The library adds a new rule:

REQ-LIB-05: A member cannot reserve more than one book at the same time.

Discuss briefly:

- Which class may need to change?
- Which test cases need to be added?
- What should be added to the RTM?

Independent Practice Task

Clinic Prescription Refill Request System

A small medical clinic receives prescription refill requests by phone and email. This creates quality problems because patient details may be incomplete, urgent requests may not be identified, duplicate requests may occur, and staff may not provide clear feedback to patients. The clinic wants a simple C# prototype that manages prescription refill requests.

Your Task

Independently create a new C# class library and MSTest project named:

- PrescriptionRefill
- PrescriptionRefill.Tests

Your prototype must support the following requirements:

Requirement ID	Requirement
REQ-RX-01	The system shall allow a valid patient to submit a refill request for a medicine name.
REQ-RX-02	The system shall reject a refill request if the patient ID is empty.
REQ-RX-03	The system shall reject a refill request if the medicine name is empty.
REQ-RX-04	The system shall mark a request as urgent if the patient has two or fewer days of medicine remaining.
REQ-RX-05	The system shall return a clear success or failure message for every request.

Required Classes

Create suitable classes such as:

- Patient
- RefillRequest
- RefillResult
- RefillService

You may design the class structure yourself, but it must support the requirements.

Required Tests

Write at least five MSTest tests, including:

- SubmitRequest_ValidPatientAndMedicine_ReturnsSuccess
- Patient_EmptyPatientId_ThrowsException
- SubmitRequest_EmptyMedicineName_ReturnsFailure
- SubmitRequest_TwoOrFewerDaysRemaining_MarksRequestAsUrgent
- SubmitRequest_ResultMessage_IsClear

Required RTM

Create: docs/RTM.md

Your RTM must link:

- requirement ID

- requirement summary
- acceptance criterion
- test case name
- status

Required Copilot Reflection

Use GitHub Copilot for support and record:

- Prompt used
- One useful suggestion
- One suggestion you modified
- One suggestion you rejected
- Why human judgement was required